

Resume Optimizer OTG

One-page repo summary based on code and README evidence only.

WHAT IT IS

A Streamlit-based career application workspace for tailoring existing resumes, generating fresh resumes, building a reusable career profile, and reviewing fit before export. The repo also includes a Flask web app and a legacy Tkinter desktop app.

WHO IT IS FOR

Primary user: a job seeker or career switcher who wants to optimize resumes for specific roles while reusing a long-term profile of experience, skills, projects, and application assets.

WHAT IT DOES

- Uploads and reads resume source files, including DOCX for optimization and PDF/DOCX/TXT/MD for profile import.
- Builds a reusable career profile from imported documents or structured notes, then stores items in SQLite.
- Cleans job descriptions from pasted text or fetched URLs, then matches saved profile evidence to the target role.
- Offers manual, API, and local/custom-endpoint run paths for structured resume optimization.
- Validates AI JSON payloads, runs pre-flight anchor review, and applies exact paragraph replacements while preserving document formatting.
- Scores resume-to-job fit with a local deterministic evaluator and surfaces recommendations, gap severity, and before/after deltas.
- Saves application workspaces and resume assets for reuse across future job targets.

HOW IT WORKS

- UI layer: ``resume_optimizer_local/streamlit_app.py`` drives the main product flow; ``resume_optimizer_web/app.py`` provides a separate Flask interface; ``resume_optimizer/main.py`` is a legacy Tkinter entry point.
- Input + extraction: ``docx_handler.py`` reads resumes and applies replacements; ``profile_extractor.py`` parses imported profile sources into basics and reusable items; ``jd_fetcher.py`` and ``jd_cleaning.py`` prepare job descriptions.
- Decision layer: ``resume_evaluator.py`` computes fit signals; ``profile_matcher.py`` ranks saved profile items against the job; ``prompt_engine.py`` builds optimizer and builder prompts.
- AI + validation: ``ai_gateway.py`` dispatches to OpenAI, Anthropic, Gemini, or OpenAI-compatible local endpoints; ``json_parser.py`` and ``review_engine.py`` validate and review replacement payloads before document mutation.
- Persistence: ``profile_store.py`` uses SQLite (``career_profile.db``) for career profiles, imported sources, profile items, applications, application-profile links, and resume assets.
- Data flow: source resume / profile docs + target job -> extraction / cleaning -> fit and evidence selection -> prompt / structured output -> validation / review -> DOCX export + asset save.

HOW TO RUN

- From the repo root: ``cd resume_optimizer_local``
- Install deps: ``python3 -m pip install -r requirements.txt``
- Start the current app: ``python3 -m streamlit run streamlit_app.py``
- Open the local Streamlit URL shown in the terminal.
- Optional alternatives in repo: Flask web app (``resume_optimizer_web/app.py``) and legacy Tkinter app (``resume_optimizer/main.py``).

REPO GAPS / NOTES

- Hosted production URL: Not found in repo.
- User authentication / multi-user account system: Not found in repo.
- External managed database service: Not found in repo; local persistence is SQLite.